EVPN-VXLAN Control-Plane Convergence Project

A plain-language explanation of the components, workflows and architecture

> **Project statement:** Reduced control plane convergence time 40% across an EVPN-VXLAN overlay fabric serving 1K+ tenant endpoints, as measured by failover benchmarks, by implementing C++ EVPN Type-2/Type-5 MAC/IP route handling with hash-based data structures and event-driven state machines.

What is inside this document

- 1. Overall summary in simple terms
- 2. Decoding every phrase of the project statement
- 3. Key terms (mini glossary)
- 4. Architecture diagrams, explained element by element
- 5. Components, explained in detail
- 6. Workflows, explained step by step from end to end
- 7. Why the result was 40% faster
- 8. Likely interview questions with short answers

1. Overall Summary (Simple Terms)

A large data center is shared by many customers, called **tenants**. Each tenant has its own virtual machines, servers or containers, called **endpoints**. In this project there were more than **1,000 endpoints**. Every endpoint needs to be reachable by the others in its tenant, no matter which physical switch it is plugged into.

To make this work, the data center uses a design called **EVPN-VXLAN**. Think of it as two layers working together:

- **VXLAN** is the delivery truck. It wraps each tenant packet inside an outer packet so it can travel across the shared physical network and be unwrapped on the other side.
- **EVPN** is the address book. It is a set of messages, carried by the routing protocol BGP, that tells every switch where each endpoint lives: "MAC address X and IP address Y are behind switch Z."

The software that keeps this address book up to date is called the **control plane**. When something breaks (a switch dies, a link fails, a VM moves), the control plane must notice, update the address book everywhere, and reprogram the switches. The time this takes is called **convergence time**. While it is happening, some traffic is lost or goes to the wrong place.

This project rewrote the part of the control plane that handles the two most important EVPN messages, **Type-2** (where a specific device lives) and **Type-5** (where a whole IP network lives). It was written in **C++** and used two key ideas:

- **Hash-based data structures:** instead of searching through a long list to find an entry, the software jumps straight to it, like looking up a word in a dictionary by its first letters rather than reading every page.
- **Event-driven state machines:** instead of waking up on a timer and re-checking everything, the software reacts the moment something happens and only touches the entries that are affected.

The outcome was measured with failover tests (deliberately breaking things and timing the recovery). Recovery became **40% faster**: if the old design needed 1.0 second to recover, the new one needed about 0.6 seconds.

> ***In simple terms:*** *Imagine a huge apartment building's mail room. The old clerk, whenever a resident moved, re-read the entire resident list every few minutes. The new clerk keeps an indexed card file and updates only the one card that changed, the instant they hear about it. Mail stops going to the wrong door much sooner.*

2. Decoding Every Phrase of the Project Statement

Phrase	What it means in simple terms
Reduced control plane convergence time	Made the network's "brain" recover faster after a failure, so the address book is correct again sooner.
40%	Recovery time dropped to about 60% of what it was before. Example: 1.0 s became 0.6 s.
EVPN-VXLAN overlay fabric	A data-center network where tenant traffic rides on top of ("overlays") a shared physical network made of spine and leaf switches (the "fabric").
Serving 1K+ tenant endpoints	More than 1,000 virtual machines, servers or containers, belonging to different customers, rely on it.
As measured by failover benchmarks	The 40% figure comes from repeatable tests that break a link or switch on purpose and time how long it takes to fully recover.
C++	The programming language used. Chosen because it is fast and gives fine control over memory.
EVPN Type-2 route	A message that says: this MAC address (and optionally this IP address) lives behind this switch.
EVPN Type-5 route	A message that says: this whole IP subnet (for example 10.20.0.0/16) is reachable through this switch.
MAC/IP route handling	Receiving, storing, updating, deleting and re-sending those Type-2 and Type-5 messages.
Hash-based data structures	Tables that find any entry almost instantly using a computed key, no matter how many entries exist.
Event-driven state machines	Each route has a small set of states (new, active, moving, being deleted). It changes state only when a specific event arrives.

3. Key Terms (Mini Glossary)

Term	Simple meaning
Fabric	The whole set of switches wired together in the data center.
Spine switch	A top-level switch. Every leaf connects to every spine. In this design spines also act as BGP route reflectors.
Leaf switch	A switch that servers plug into. Each leaf is also a VTEP.
VTEP	VXLAN Tunnel End Point. The device that wraps (encapsulates) and unwraps (decapsulates) VXLAN packets. Identified by a loopback IP address.
Underlay	The real physical IP network between switches. It only needs to know how to reach each VTEP's loopback IP.
Overlay	The virtual tenant networks that ride on top of the underlay inside VXLAN tunnels.
VNI	VXLAN Network Identifier. A 24-bit number (about 16 million possible values) that says which virtual network a packet belongs to.
L2 VNI / L3 VNI	An L2 VNI is a virtual LAN segment (bridging). An L3 VNI represents a tenant's routing table (VRF) for routing between subnets.
VRF	Virtual Routing and Forwarding. A separate routing table per tenant, so tenants cannot see each other.
MAC address	A device's hardware address, for example 00:1a:2b:3c:4d:5e. Used for Layer-2 delivery.
IP address	A device's network address, for example 192.168.10.11. Used for Layer-3 routing.
BGP	Border Gateway Protocol. The routing protocol used to carry EVPN messages between switches.
Route Reflector (RR)	A BGP speaker (here, a spine) that receives routes from each leaf and reflects them to all other leaves, so leaves do not need a session with each other.
NLRI	Network Layer Reachability Information. The "payload" of a BGP update, here the EVPN route itself.
RD / RT	Route Distinguisher makes a route unique. Route Target is a tag that decides which tenant's table should import the route.
ESI	Ethernet Segment Identifier. Identifies a server connected to two or more leaves at once (multihoming).
FIB / ASIC	FIB is the forwarding table the hardware uses. The ASIC is the switch chip that forwards packets at line rate.

Term	Simple meaning
BFD	Bidirectional Forwarding Detection. A very fast heartbeat that detects a dead link or neighbor in milliseconds.
Convergence	The process of every switch reaching a correct and consistent view of the network after a change.
FSM	Finite State Machine. A model with a fixed set of states and clear rules for moving between them.
Hash table	A table that turns a key into a slot number using a hash function, so lookups take roughly constant time.

4. Architecture Diagrams, Explained Element by Element

The architecture is shown in three diagrams. **Figure 1** is the zoomed-out view of the whole fabric. **Figure 2** zooms into a single leaf switch and shows the C++ software built in this project. **Figure 3** shows the failover timeline before and after the change.

4.1 Figure 1: The Whole Fabric

Figure 1 - EVPN-VXLAN Fabric: Physical and Logical View

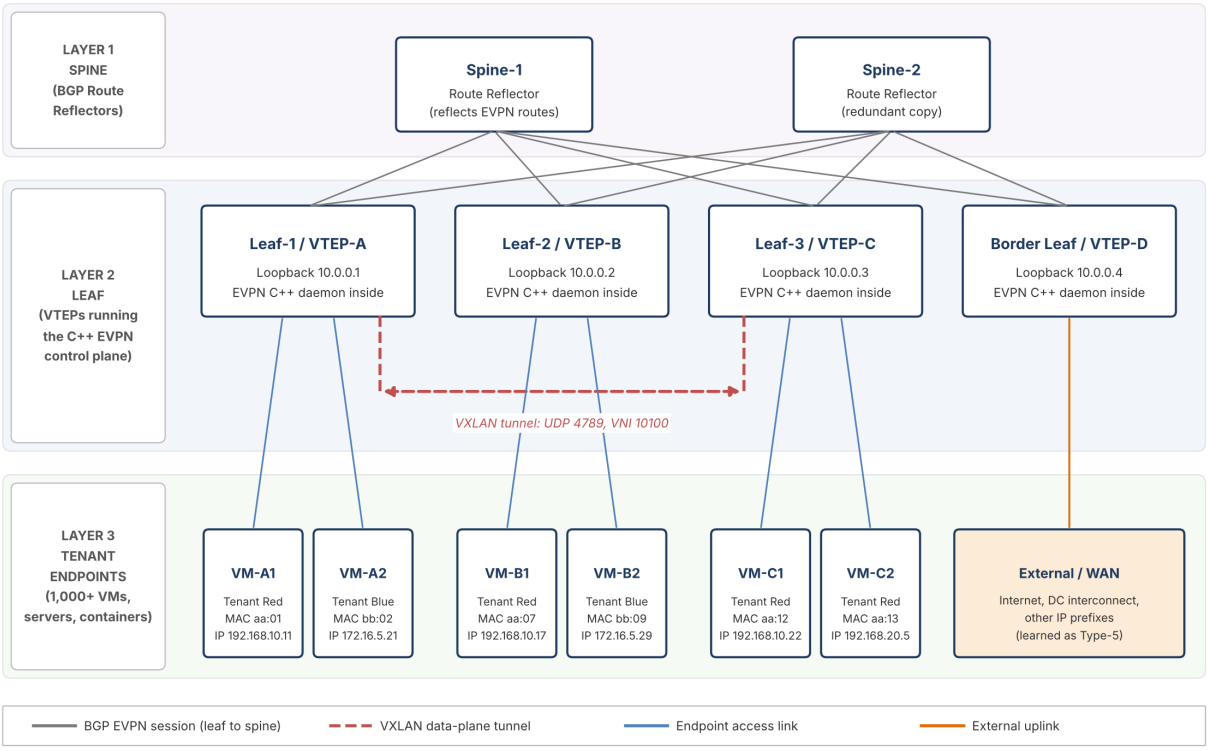


Figure 1. Spine-leaf EVPN-VXLAN fabric with tenant endpoints and an external connection.

Layer 1: Spine (top band)

- **Spine-1 and Spine-2** are the top-level switches. Every leaf has a physical cable to each of them.
- They act as **BGP Route Reflectors**. When Leaf-1 announces "VM-A1 is behind me," it sends that once to the spines, and the spines forward (reflect) it to every other leaf.
- There are two spines for **redundancy**. If one dies, the other still reflects every route, so the address book keeps working.
- The spines do not need to know about tenant MACs for forwarding. They just move outer VXLAN packets between leaf loopback IPs.

Grey lines (between spine and leaf)

- Each grey line is both a physical uplink and an **iBGP EVPN session** (TCP port 179).

- With 2 spines and 4 leaves there are 8 sessions. Each leaf has two, which is why it still learns routes if one spine fails.

Layer 2: Leaf (middle band)

- **Leaf-1, Leaf-2, Leaf-3** connect tenant endpoints. **Border Leaf** connects the fabric to the outside world.
- Each leaf is a **VTEP** with a unique **loopback IP** (10.0.0.1 to 10.0.0.4). That IP is the "street address" used as the outer destination for VXLAN packets.
- "**EVPN C++ daemon inside**" means each leaf runs the software built in this project. Figure 2 shows what is inside that daemon.

Red dashed line (VXLAN tunnel)

- Shows a **data-plane tunnel** from Leaf-1 to Leaf-3. When VM-A1 talks to VM-C1 (both Tenant Red), Leaf-1 wraps the packet in VXLAN and sends it to 10.0.0.3.
- **UDP 4789** is the standard VXLAN port. **VNI 10100** is the example virtual network ID for Tenant Red's subnet.
- Physically the tunnel packets travel Leaf-1 to a spine to Leaf-3. The dashed line is the logical path, not a separate cable.
- Tunnels are not configured one by one. A leaf builds a tunnel to another VTEP automatically after it learns a Type-2 or Type-5 route pointing at that VTEP.

Layer 3: Tenant endpoints (bottom band)

- Each box (VM-A1, VM-A2 and so on) is one endpoint. Each one shows its **tenant**, its **MAC** and its **IP**.
- **Tenant Red** and **Tenant Blue** are isolated from each other. Red VMs use VNI 10100 and their own VRF. Blue VMs use a different VNI and VRF.
- VM-C2 is on a different Red subnet (192.168.20.0/24). Reaching it from VM-A1 needs routing, which uses the L3 VNI and Type-2 routes that carry an IP, or Type-5 routes for the subnet.
- The real fabric had **1,000+** endpoints. Six are drawn to keep the picture readable.

Blue lines (access links)

- The cable or virtual port between an endpoint and its leaf. When an endpoint sends its first packet, the leaf learns its MAC (and IP via ARP) on this link.

External / WAN box and orange line

- Represents the internet, other data centers or other networks. The Border Leaf learns these networks and announces them inside the fabric as **Type-5 IP prefix routes**.

Legend (bottom strip)

- Explains the four line styles so the picture can be read without the text.

4.2 Figure 2: Inside One Leaf (the Software Built in This Project)

Figure 2 - Inside One Leaf: C++ EVPN Control-Plane Architecture

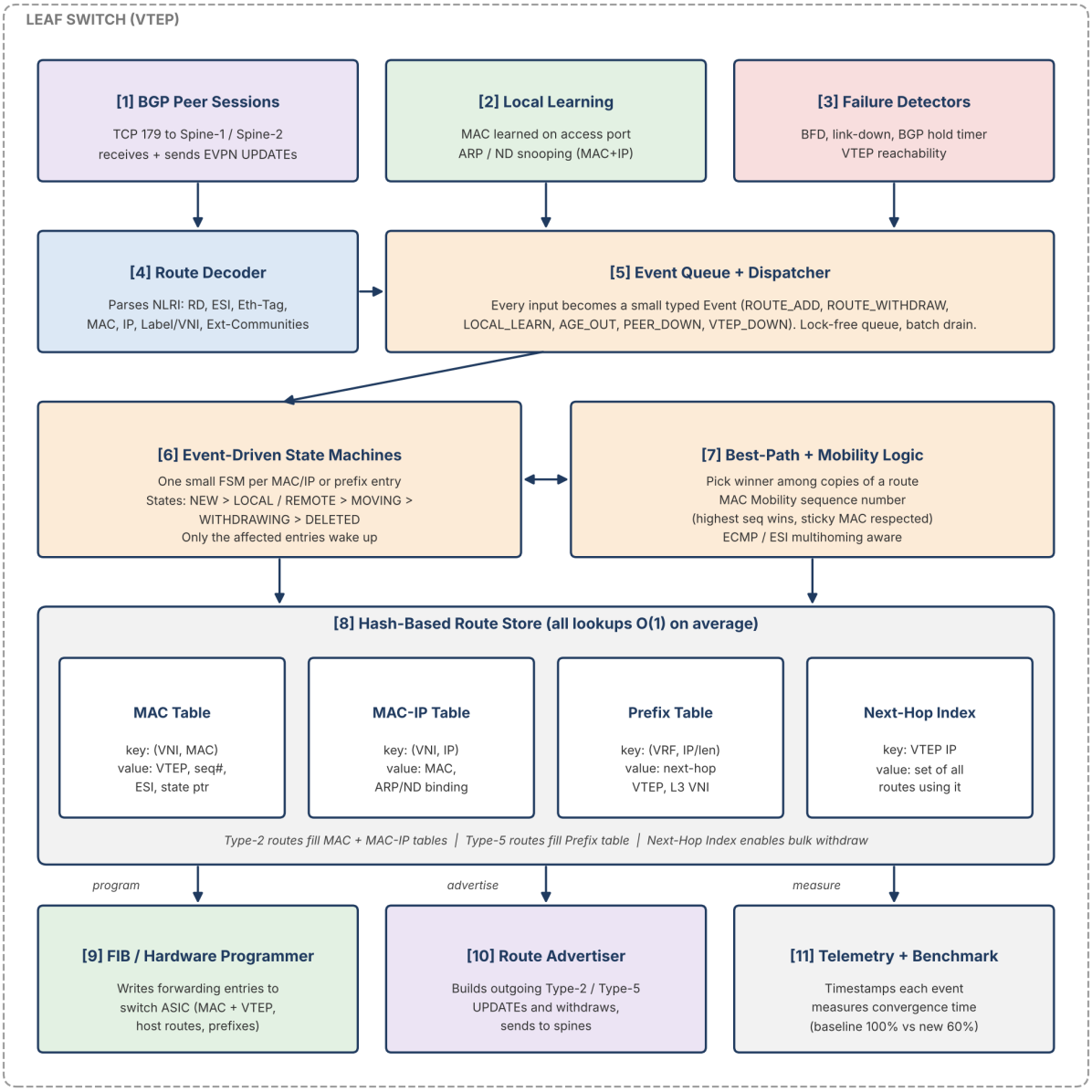


Figure 2. Modules of the C++ EVPN control-plane daemon running on each leaf.

The diagram reads from **top to bottom**: inputs at the top, processing in the middle, storage in the hash tables, and outputs at the bottom. Each numbered block is explained below; Section 5 then goes deeper into each one.

Block	What it does (simple terms)	Arrow out of it
Dashed outline "LEAF SWITCH (VTEP)"	Everything inside runs on a single leaf. Every leaf in Figure 1 has its own copy.	None

Block	What it does (simple terms)	Arrow out of it
[1] BGP Peer Sessions	Keeps the TCP 179 connections to both spines alive. Receives EVPN updates from them and sends this leaf's updates to them.	Down to [4]: raw bytes of received EVPN routes.
[2] Local Learning	Notices devices plugged into this leaf: learns MACs from packets and IPs from ARP/ND snooping.	Down to [5]: a LOCAL_LEARN or AGE_OUT event.
[3] Failure Detectors	Watches for problems: BFD timeouts, links going down, BGP hold timers expiring, a remote VTEP becoming unreachable.	Down to [5]: PEER_DOWN or VTEP_DOWN event.
[4] Route Decoder	Turns raw BGP bytes into a clean C++ object with every field: RD, ESI, Ethernet Tag, MAC, IP, VNI, and extended communities.	Right to [5]: ROUTE_ADD or ROUTE_WITHDRAW event.
[5] Event Queue + Dispatcher	A single line where every event waits its turn. Lock-free and drained in batches, so bursts of thousands of events are handled efficiently.	Down to [6]: each event goes to the right state machine.
[6] Event-Driven State Machines	Each MAC/IP or prefix has its own tiny FSM. Only the FSMs touched by an event run.	Two-way with [7]; down to [8] to update the stored entry.
[7] Best-Path + Mobility Logic	If two leaves claim the same MAC, decides who wins (highest MAC Mobility sequence number). Also handles multihomed servers (ESI) and equal-cost paths.	Two-way with [6]; down to [8] to record the winner.
[8] Hash-Based Route Store	Four hash tables: MAC Table, MAC-IP Table, Prefix Table and Next-Hop Index. Every lookup is about constant time.	Down to [9], [10] and [11].
MAC Table	Key (VNI, MAC). Value: which VTEP, sequence number, ESI, pointer to its FSM. Filled by Type-2.	Part of [8]
MAC-IP Table	Key (VNI, IP). Value: the MAC bound to that IP. Filled by Type-2 routes carrying an IP. Used for ARP suppression and host routing.	Part of [8]
Prefix Table	Key (VRF, prefix/length). Value: next-hop VTEP and L3 VNI. Filled by Type-5.	Part of [8]
Next-Hop Index	Key: a VTEP IP. Value: the set of every route that points to that VTEP. This is what makes failover fast: one lookup finds everything to remove.	Part of [8]
[9] FIB / Hardware Programmer	Writes final forwarding entries into the switch chip so real packets go the right way.	Program (into the ASIC)
[10] Route Advertiser	Builds and sends this leaf's own Type-2 and Type-5 updates and withdrawals to the spines.	Advertise (back out through [1])

Block	What it does (simple terms)	Arrow out of it
[11] Telemetry + Benchmark	Timestamps every event and change to measure convergence. Produced the before/after numbers.	Measure (reports)

4.3 Figure 3: Failover Timeline, Old vs New

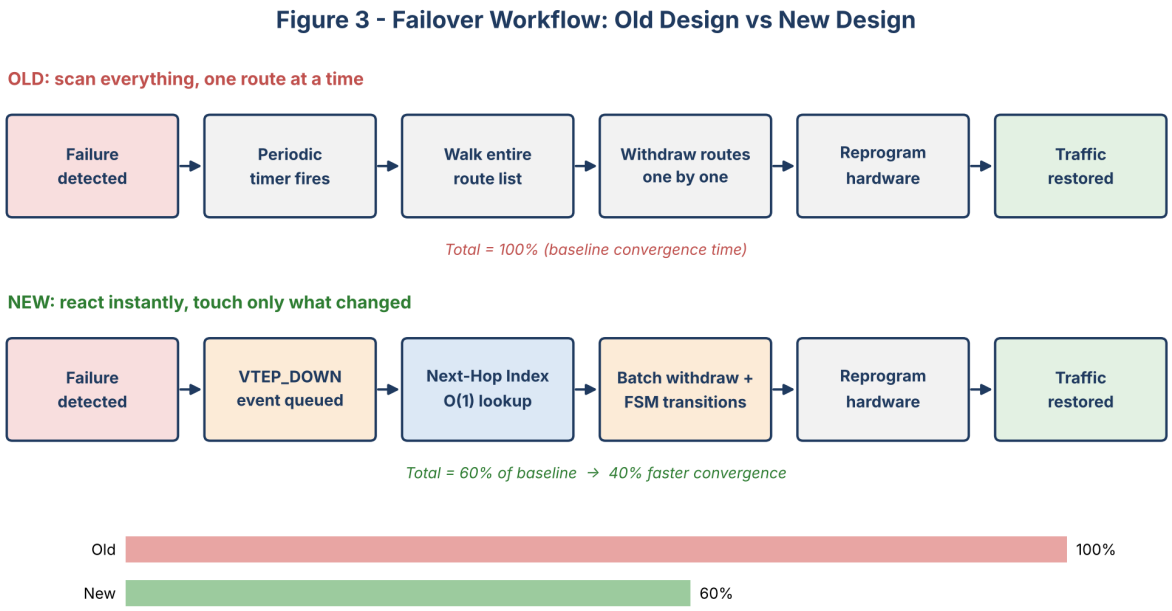


Figure 3. The same failure handled by the old design (top) and the new design (bottom).

Top row (old design)

- **Failure detected:** a leaf or link goes down.
- **Periodic timer fires:** the old code did not react right away. It waited for a scan timer, which added idle delay.
- **Walk entire route list:** it looped through every stored route to find the ones using the failed VTEP. With 1,000+ endpoints this is slow and gets slower as the fabric grows.
- **Withdraw routes one by one:** each affected route was removed and re-announced separately, creating many small messages and hardware writes.
- **Reprogram hardware**, then **traffic restored**. The total is the 100% baseline.

Bottom row (new design)

- **Failure detected:** same trigger (BFD or link-down).
- **VTEP_DOWN event queued:** an event is created immediately, with no waiting on a timer.
- **Next-Hop Index O(1) lookup:** a single hash lookup returns exactly the routes that used the dead VTEP.

- **Batch withdraw + FSM transitions:** only those routes' state machines run, and their changes are grouped into a few large updates instead of many small ones.
- **Reprogram hardware**, then **traffic restored**. The total is about 60% of baseline.

Bars at the bottom

- A visual comparison: the green bar (new) is 60% as long as the red bar (old). That gap is the **40% reduction**.

5. Components, Explained in Detail

Components are grouped into two parts: the **network building blocks** (what already existed in the fabric) and the **software modules** (what this project built in C++).

Part A: Network building blocks

A1. Spine-leaf fabric

- **What it is:** a two-tier network. Every leaf connects to every spine; leaves never connect to each other directly.
- **Why it is used:** any server is at most two hops (leaf to spine to leaf) from any other, giving predictable latency and many equal paths.
- **Its role here:** provides the underlay IP network that carries VXLAN packets and the BGP sessions that carry EVPN routes.

> ***In simple terms:*** *Spines are the main highways; leaves are the on-ramps where neighborhoods (servers) connect.*

A2. Underlay network

- A plain IP network (often OSPF, IS-IS or eBGP) whose only job is to make every VTEP loopback IP reachable from every other.
- It knows nothing about tenants, MACs or VMs. That is what keeps it simple and stable.

> ***In simple terms:*** *The road system. It knows how to get to each building, not who lives in each apartment.*

A3. VXLAN (data plane)

- **Encapsulation:** the original Ethernet frame from the VM is wrapped with a VXLAN header (8 bytes, holding the 24-bit VNI), then a UDP header (destination port 4789), an outer IP header (source = local VTEP, destination = remote VTEP) and an outer Ethernet header.
- **Overhead:** about 50 bytes, so the underlay MTU is usually raised (for example to 9,000 or at least 1,550 bytes).
- **Decapsulation:** the remote VTEP removes the outer headers, reads the VNI to know which tenant network it belongs to, and delivers the original frame.
- **Entropy:** the outer UDP source port is derived from a hash of the inner packet, so different flows spread across the different spine paths.

> ***In simple terms:*** *Putting a letter (the VM's packet) inside a shipping envelope addressed from one building to another, with a tenant code on the label.*

A4. BGP EVPN (control plane protocol)

- BGP is extended with the **L2VPN EVPN address family** (AFI 25, SAFI 70) so it can carry MAC and IP reachability instead of only IP prefixes.
- Leaves peer with the spine route reflectors using iBGP. A route sent once by a leaf reaches every other leaf through the spines.
- **Route Targets** control which VRF or VNI imports a route, which keeps tenants separate.
- EVPN defines several route types. This project focused on **Type-2** and **Type-5**, the ones that carry endpoint and subnet reachability and therefore dominate failover work.

> ***In simple terms:*** *A gossip network for addresses. Each switch tells the spines who lives behind it, and the spines tell everyone else.*

A5. EVPN Type-2: MAC/IP Advertisement route

Announces that a specific endpoint lives behind a specific VTEP. Its fields:

Field	Size	Simple meaning
Route Distinguisher (RD)	8 bytes	Makes the route unique per leaf and VNI, for example 10.0.0.1:100.
Ethernet Segment Identifier (ESI)	10 bytes	Zero for a single-homed host. Non-zero if the host is connected to two leaves.
Ethernet Tag ID	4 bytes	Identifies the broadcast domain; usually 0 in common VXLAN setups.
MAC Address Length + MAC	1 + 6 bytes	The endpoint's MAC address (length is 48 bits).
IP Address Length + IP	1 + 0/4/16 bytes	Optional. The endpoint's IPv4 or IPv6 address, if known from ARP/ND.
Label 1	3 bytes	Carries the L2 VNI (for bridging, same subnet).
Label 2	0 or 3 bytes	Carries the L3 VNI (for routing between subnets), when an IP is present.
Next hop (BGP attribute)	4 bytes	The VTEP loopback IP where this endpoint lives.
Extended communities	varies	Route Target, Encapsulation = VXLAN, MAC Mobility (sequence number, sticky flag), Router MAC.

- **MAC-only Type-2** fills the MAC Table (Layer-2 bridging).
- **MAC+IP Type-2** fills both the MAC Table and the MAC-IP Table, enabling ARP suppression (the leaf answers ARP locally) and host routing.

> ***In simple terms:*** *A change-of-address card: "Device with MAC aa:01 and IP 192.168.10.11, in network 10100, now lives at building 10.0.0.1."*

A6. EVPN Type-5: IP Prefix route

Announces that a whole IP network is reachable through a VTEP. Its fields:

Field	Size	Simple meaning
Route Distinguisher	8 bytes	Makes the route unique.
ESI	10 bytes	Usually zero.
Ethernet Tag ID	4 bytes	Usually zero.
IP Prefix Length + Prefix	1 + 4/16 bytes	The network, for example 10.20.0.0/16 or 0.0.0.0/0 (default route).
Gateway IP	4/16 bytes	Often zero in VXLAN; the next hop is the VTEP itself.
Label	3 bytes	Carries the tenant's L3 VNI.
Extended communities	varies	Route Target, Encapsulation = VXLAN, Router MAC of the advertising VTEP.

- Used for subnets that are not tied to a single MAC: external networks behind the Border Leaf, summarized subnets, or routes between tenant VRFs.

> ***In simple terms:*** *Instead of listing every house, a sign that says "all of Elm Street is reached through gate D."*

A7. Tenants, VNIs and VRFs

- Each tenant gets one or more **L2 VNIs** (one per subnet) and one **L3 VNI** tied to its **VRF**.
- Routes are tagged with Route Targets so they are imported only into the right tenant's tables. Tenant Red never sees Tenant Blue's routes.
- 1,000+ endpoints across tenants means thousands of Type-2 routes (one MAC route and often one MAC+IP route per endpoint) plus the Type-5 prefixes.

Part B: Software modules built in C++

B1. BGP Peer Session handler [block 1]

- Maintains TCP connections (port 179) to both spines and runs the BGP session state (Idle, Connect, OpenSent, OpenConfirm, Established).
- Reads incoming UPDATE messages and passes the EVPN part (MP_REACH / MP_UNREACH attributes) to the Route Decoder.
- Sends KEEPALIVEs and tracks the hold timer. If a spine goes silent, it raises PEER_DOWN.
- Uses non-blocking sockets driven by an event loop (for example epoll), so one slow peer never blocks processing.

> ***In simple terms:*** *The phone line to the spines, kept open all the time.*

B2. Local Learning [block 2]

- When a packet arrives on an access port from an unknown source MAC, the hardware reports it. The module creates a LOCAL_LEARN event.
- **ARP/ND snooping:** watches ARP (IPv4) and Neighbor Discovery (IPv6) packets to learn which IP belongs to which MAC.
- **Aging:** if a MAC is silent for too long, an AGE_OUT event removes it and triggers a withdrawal.

> ***In simple terms:*** *The doorman who notes the name of every new resident who walks in.*

B3. Failure Detectors [block 3]

- **BFD:** sends tiny heartbeat packets every few tens or hundreds of milliseconds. Missing several in a row means the neighbor is dead, much faster than the BGP hold timer (often 90 or more seconds by default).
- **Link-down signal:** the fastest signal, straight from the port hardware.
- **VTEP reachability:** if the underlay loses the route to a remote VTEP's loopback IP, that VTEP is treated as unreachable and a VTEP_DOWN event is raised.

> ***In simple terms:*** *Smoke alarms. They do not fix the fire; they shout the moment they notice it.*

B4. Route Decoder [block 4]

- Parses the EVPN NLRI byte by byte using the field layouts in A5 and A6.
- Validates lengths (for example, the MAC length must be 48 bits; the IP length must be 0, 32 or 128 bits) and rejects malformed routes safely.
- Extracts extended communities: Route Targets (which tenant), MAC Mobility sequence number and sticky flag, Router MAC, encapsulation type.
- Produces a compact C++ struct (for example a Type2Route or Type5Route) and wraps it in a ROUTE_ADD or ROUTE_WITHDRAW event.
- Avoids extra copying by parsing directly from the receive buffer, so large bursts of updates stay fast.

> ***In simple terms:*** *A translator that turns a compressed telegram into a neat form with labeled boxes.*

B5. Event Queue + Dispatcher [block 5]

- Every input, from any source, becomes a small typed **event**: ROUTE_ADD, ROUTE_WITHDRAW, LOCAL_LEARN, AGE_OUT, PEER_DOWN, VTEP_DOWN.
- Events go into a **lock-free queue**, so producers (network, hardware, detectors) never wait on each other.
- The dispatcher **drains events in batches**. If 500 withdrawals arrive at once, they are processed together and the resulting hardware and BGP updates are grouped.
- Events for the same key (for example the same MAC) are kept in order, so an add followed by a withdraw is never processed backwards.

- **Coalescing:** if a route is added and withdrawn within the same batch, the net result is computed once instead of doing two pieces of work.

> ***In simple terms:*** *A single, well-organized ticket line instead of everyone shouting at the clerk at once.*

B6. Event-Driven State Machines [block 6]

Each MAC, MAC-IP or prefix entry has its own small FSM. The states:

State	Meaning	Typical events that move it
NEW	Just created, not yet decided.	LOCAL_LEARN goes to LOCAL; ROUTE_ADD goes to REMOTE.
LOCAL	Endpoint is directly attached to this leaf.	AGE_OUT goes to WITHDRAWING; a remote ROUTE_ADD with higher sequence goes to MOVING.
REMOTE	Endpoint lives behind another VTEP.	ROUTE_WITHDRAW or VTEP_DOWN goes to WITHDRAWING; LOCAL_LEARN goes to MOVING.
MOVING	Endpoint is changing location (VM migration). Mobility logic decides the winner.	Decision made goes to LOCAL or REMOTE.
WITHDRAWING	Being removed from hardware and, if local, withdrawn from BGP.	Cleanup done goes to DELETED; a new ROUTE_ADD can revive it to REMOTE.
DELETED	Gone. Memory is reclaimed.	Terminal state.

- **Why it helps:** nothing is polled. Only entries that receive an event do any work, so a failure that affects 200 routes costs 200 small transitions, not a scan of 5,000 entries.
- **Correctness:** every transition is explicit, so odd sequences (add, add, withdraw, add) always land in a well-defined state. This removed a class of bugs where stale routes lingered.
- **Implementation detail:** a transition table (state x event gives action + next state), with the entry pointing to its FSM so no extra lookup is needed.

> ***In simple terms:*** *Each address card has a colored sticker (new, here, elsewhere, moving, leaving). The clerk changes the sticker only when a specific thing happens to that card.*

B7. Best-Path + Mobility Logic [block 7]

- **Duplicate copies:** a route arrives from both spines (two reflected copies). They are merged into one entry.
- **MAC Mobility:** when a VM moves from Leaf-1 to Leaf-3, Leaf-3 advertises the MAC with a **higher sequence number**. Everyone prefers the highest number, so the old location loses and Leaf-1 withdraws its stale route.
- **Sticky (static) MACs** are never allowed to move, which protects gateways and critical devices.

- **Duplicate detection:** if a MAC flips back and forth too many times in a short window (for example 5 moves in 180 seconds), it is flagged to stop a loop or a misconfiguration from flooding the fabric.
- **Multihoming (ESI) and ECMP:** if a server is connected to two leaves, both paths are kept and traffic is shared. If one fails, the other is used immediately.

> ***In simple terms:*** *A referee who decides which address is newest when two buildings both claim the same resident.*

B8. Hash-Based Route Store [block 8]

- Built with C++ hash maps (for example std::unordered_map with custom hash functions, or an open-addressing table for better cache use).
- **Composite keys:** (VNI, MAC), (VNI, IP), (VRF, prefix). A custom hash mixes the fields so entries spread evenly and collisions are rare.
- **Pre-sizing (reserve):** the tables are sized up front for the expected number of entries so they do not need to resize (rehash) in the middle of a failover.
- **Next-Hop Index:** a reverse map from VTEP IP to the set of routes that use it. This is the single biggest win for failover: finding every route behind a dead VTEP is one lookup instead of a full scan.
- **Prefix lookups:** exact-match on (VRF, prefix) is a hash lookup. Longest-prefix-match for forwarding is done by the hardware FIB, so the software only needs exact matches.

Operation	Old (list / tree scan)	New (hash)
Find one MAC	Grows with table size (O(n) or O(log n))	About constant (O(1) average)
Find all routes behind a VTEP	Scan everything (O(n))	One lookup + walk only affected (O(k))
Insert or delete a route	O(log n) or O(n)	About constant (O(1) average)

> ***In simple terms:*** *A dictionary with tabs on the side, plus a second index that lists, for each building, every resident in it.*

B9. FIB / Hardware Programmer [block 9]

- Converts the winning routes into hardware entries: a remote MAC becomes "MAC X in VNI Y goes into a tunnel to VTEP Z"; a host or prefix route becomes "IP prefix in VRF goes to VTEP Z with L3 VNI".
- **Batches writes** to the switch chip, because many small writes are much slower than a few large ones.
- **Next-hop groups:** many routes share the same tunnel next hop. When a VTEP fails, changing the shared group updates all those routes at once in hardware.

- Confirms each write and retries on error, so software and hardware never disagree.

> ***In simple terms:*** *The person who actually moves the signposts on the road once the address book has been updated.*

B10. Route Advertiser [block 10]

- Builds this leaf's own Type-2 routes for locally learned endpoints and Type-5 routes for local or external subnets.
- Sends withdrawals when local endpoints age out or move away.
- Packs many routes into each BGP UPDATE message (one shared set of attributes, many NLRIs), reducing message count during bursts.
- Bumps the MAC Mobility sequence number when it advertises a MAC that was previously remote.

> ***In simple terms:*** *The post office that mails this building's change-of-address cards to the spines.*

B11. Telemetry + Benchmark harness [block 11]

- Records high-resolution timestamps at each stage: detected, queued, decided, programmed, advertised.
- Runs repeatable **failover benchmarks**: shut a leaf, cut an uplink, kill a BGP session, migrate VMs; then measure when traffic fully recovers.
- Uses a traffic generator sending a known packet rate between endpoints. **Convergence time = packets lost / packet rate.** For example, 60,000 packets lost at 100,000 packets per second means 0.6 s.
- Each test is repeated many times and reported as median and tail (p95/p99) so the 40% result is not a lucky single run.

> ***In simple terms:*** *A stopwatch that starts when something breaks and stops when traffic is flowing correctly again.*

6. Workflows, Explained Step by Step

Each workflow uses the example names from Figure 1. Block numbers in brackets refer to Figure 2.

Workflow 1: A new VM comes online (local learning and advertisement)

Step 1. VM-A1 powers on on a server behind Leaf-1. It sends its first packet, often an ARP request ("who has 192.168.10.1?").

Step 2. Leaf-1 hardware sees an unknown source MAC aa:01 on its access port in VLAN mapped to VNI 10100. It reports this to the software.

Step 3. Local Learning [2] creates LOCAL_LEARN(VNI 10100, MAC aa:01). ARP snooping also notes that IP 192.168.10.11 belongs to aa:01.

Step 4. The event enters the Event Queue [5]. The dispatcher picks it up in the next batch, usually within microseconds.

Step 5. Hash lookup in the MAC Table [8] by key (10100, aa:01). Not found, so a new entry is created with an FSM in state NEW.

Step 6. FSM [6] moves NEW to LOCAL. Mobility logic [7] checks there is no remote copy with a higher sequence number.

Step 7. The entry is also added to the MAC-IP Table [8] with key (10100, 192.168.10.11).

Step 8. Route Advertiser [10] builds a Type-2 route: RD 10.0.0.1:100, MAC aa:01, IP 192.168.10.11, L2 VNI 10100, L3 VNI of Tenant Red, next hop 10.0.0.1, Route Target for Tenant Red, encapsulation VXLAN.

Step 9. BGP Peer Sessions [1] send the UPDATE to Spine-1 and Spine-2.

Step 10. Both spines reflect it to Leaf-2, Leaf-3 and the Border Leaf. Each of them runs Workflow 2.

> **Result:** within a fraction of a second, every leaf knows that aa:01 / 192.168.10.11 lives behind 10.0.0.1.

Workflow 2: A leaf receives a remote route

Step 1. Leaf-3's BGP session [1] receives the UPDATE from Spine-1, and a moment later the same route from Spine-2.

Step 2. Route Decoder [4] parses the NLRI, validates field lengths, and reads the Route Target.

Step 3. Import check: the Route Target matches Tenant Red, which exists on Leaf-3, so the route is accepted. If no local tenant matches, it is ignored.

Step 4. A ROUTE_ADD event is queued [5].

Step 5. Hash lookup (10100, aa:01) in the MAC Table [8]: new entry, FSM NEW to REMOTE.

Step 6. The second copy (from Spine-2) arrives. Lookup finds the existing entry; best-path [7] sees it is the same route and simply records the extra source. No new work is done.

Step 7. Next-Hop Index [8]: the route is added to the set under key 10.0.0.1.

Step 8. FIB Programmer [9] writes: "in VNI 10100, MAC aa:01 goes into the VXLAN tunnel to 10.0.0.1", plus a host route 192.168.10.11/32 in Tenant Red's VRF using the L3 VNI.

Step 9. The ARP suppression cache is filled, so if VM-C1 asks "who has 192.168.10.11?", Leaf-3 answers locally instead of flooding the fabric.

Workflow 3: A packet travels from VM-C1 to VM-A1 (data plane)

Step 1. VM-C1 (192.168.10.22) sends a frame to MAC aa:01.

Step 2. Leaf-3 looks up (VNI 10100, aa:01) in hardware. Result: remote, tunnel to 10.0.0.1.

Step 3. Leaf-3 encapsulates: adds VXLAN header (VNI 10100), UDP (destination 4789, source port from a flow hash), outer IP (10.0.0.3 to 10.0.0.1), outer Ethernet.

Step 4. The underlay routes the outer packet Leaf-3 to a spine to Leaf-1. The spine only looks at the outer IP.

Step 5. Leaf-1 decapsulates, reads VNI 10100, and delivers the original frame to VM-A1's port.

> If VM-C1 were on a different subnet (for example VM-C2 on 192.168.20.0/24 talking to VM-A1), Leaf-3 would route it in Tenant Red's VRF and send it with the **L3 VNI** instead. Leaf-1 would route it again into the correct subnet. This is called **symmetric IRB**.

Workflow 4: A VM moves (MAC mobility)

Step 1. VM-A1 is live-migrated from Leaf-1 to Leaf-3.

Step 2. The VM sends a gratuitous ARP from its new location. Leaf-3's Local Learning [2] raises LOCAL_LEARN for aa:01.

Step 3. Leaf-3 looks up (10100, aa:01): the entry exists in state REMOTE (behind 10.0.0.1) with sequence 0.

Step 4. FSM goes REMOTE to MOVING. Mobility logic [7] makes Leaf-3 the new owner with sequence 1.

Step 5. FSM goes MOVING to LOCAL. Leaf-3 updates hardware to point aa:01 at its local port, updates the Next-Hop Index (removes from 10.0.0.1's set).

Step 6. Leaf-3 advertises Type-2 for aa:01 with MAC Mobility sequence 1.

Step 7. Leaf-1 receives it. Its local entry has sequence 0, which is lower, so it loses. Leaf-1's FSM goes LOCAL to MOVING to REMOTE, and Leaf-1 withdraws its own old Type-2.

Step 8. All other leaves see the higher sequence and repoint aa:01 to 10.0.0.3. Only one entry is touched per leaf.

Workflow 5: An external network is learned (Type-5)

Step 1. **The Border Leaf learns 10.20.0.0/16 from the external router** (for example via eBGP), inside Tenant Red's VRF.

Step 2. **Route Advertiser [10] on the Border Leaf builds a Type-5 route:** prefix 10.20.0.0/16, L3 VNI of Tenant Red, Router MAC of the Border Leaf, next hop 10.0.0.4.

Step 3. **The spines reflect it to all leaves.**

Step 4. **Each leaf decodes it [4], queues ROUTE_ADD [5],** and stores it in the Prefix Table [8] with key (Red VRF, 10.20.0.0/16). It is also added to the Next-Hop Index under 10.0.0.4.

Step 5. **FIB Programmer [9] installs the prefix route** so any Red VM sending to 10.20.x.x is routed in a VXLAN tunnel to the Border Leaf using the L3 VNI.

Workflow 6: Failover, when Leaf-1 dies (the measured scenario)

This is the workflow the 40% improvement was measured on. Suppose Leaf-1 had 250 endpoints, so other leaves hold about 500 Type-2 routes (MAC and MAC+IP) pointing at 10.0.0.1.

Step 1. **Leaf-1 loses power.** The traffic generator starts counting lost packets.

Step 2. **Detection:** on every other leaf, the underlay route to 10.0.0.1 disappears, and/or BFD to the spines' view of Leaf-1 times out. Failure Detector [3] raises VTEP_DOWN(10.0.0.1) immediately; no waiting on a scan timer.

Step 3. **Event Queue [5]** puts VTEP_DOWN at the front of the next batch.

Step 4. **One hash lookup in the Next-Hop Index [8] for key 10.0.0.1** returns the exact set of about 500 affected routes. Nothing else in the 1,000+ entry tables is touched.

Step 5. **For each affected entry, its FSM [6] moves REMOTE to WITHDRAWING.** If a backup path exists (for example a multihomed server also attached to Leaf-2), best-path [7] switches to the backup instead of deleting.

Step 6. **FIB Programmer [9] applies the changes in one batch.** Where routes shared a next-hop group, a single group update redirects or removes them all at once.

Step 7. **Shortly after, BGP withdrawals for Leaf-1's routes arrive** from the spines (once they notice the session is down). They are found already in WITHDRAWING, so the FSMs finish them to DELETED without repeating work.

Step 8. **Traffic to surviving endpoints and backup paths is fully restored.** The traffic generator stops counting loss. Telemetry [11] records each timestamp.

> **Old design for the same event:** wait for a scan timer, walk all routes comparing next hops, withdraw and reprogram each route individually. Same end result, but about 1.67 times longer.

Workflow 7: How the benchmark was run and measured

Step 1. **Set up the fabric** with 1,000+ endpoints spread across tenants, with steady-state traffic flowing between many endpoint pairs.

Step 2. **Choose a failure** (leaf power-off, uplink cut, spine reload, BGP session reset, mass VM move).

Step 3. **Trigger the failure** and let the traffic generator count lost packets per flow.

Step 4. **Compute convergence:** lost packets divided by packet rate gives the outage time for each flow; the worst flow sets the convergence time.

Step 5. **Cross-check with internal telemetry** (detect, decide, program, advertise timestamps) to see which stage got faster.

Step 6. **Repeat many times** for both old and new builds, under identical conditions, and compare medians and tail values.

Step 7. **Report:** median convergence time dropped by about 40%.

7. Why the Result Was 40% Faster

No single trick produced the 40%. It came from removing waiting and wasted work at each stage of the failover path. The table below is an illustrative breakdown of where the time went (baseline = 100 units).

Stage	Old design	New design	What changed
Notice the failure and start work	Waited for a periodic scan timer	Event fires immediately	Event-driven model removes idle waiting.
Find affected routes	Full scan of every route	One Next-Hop Index lookup	Hash-based reverse index.
Decide new state per route	Re-evaluated broadly, ad-hoc flags	Only affected FSMs run	Event-driven state machines.
Handle duplicate / late messages	Repeated work for each copy	Already-in-state checks skip work	Explicit FSM states.
Program hardware	Many single writes	Batched writes and shared next-hop groups	Batching in the dispatcher and FIB programmer.
Advertise changes	Many small BGP UPDATEs	Packed UPDATEs	Batching in the advertiser.
Overall (illustrative)	100 units	about 60 units	40% reduction in convergence time.

Why C++ specifically

- Predictable speed with no garbage-collector pauses in the middle of a failover.
- Control over memory layout (compact structs, pre-sized tables) so lookups stay in the CPU cache.
- Easy integration with existing switch SDKs and networking libraries, which are usually C or C++.

8. Likely Interview Questions with Short Answers

Question	Short answer
What is convergence time?	The time from a network change (like a failure) until every switch has a correct, consistent view and traffic flows correctly again.
Type-2 vs Type-5?	Type-2 advertises a single host (MAC, optionally its IP). Type-5 advertises a whole IP prefix, decoupled from any MAC.
Why hash tables instead of a tree or list?	Average constant-time lookup, insert and delete. The access pattern is exact-match on composite keys, so ordering (what trees give you) is not needed.
What was the single biggest win?	The Next-Hop Index: a reverse map from VTEP to routes. Failover became one lookup plus work only on the affected routes, instead of a full scan.
Why event-driven state machines?	No polling delay, work proportional to what changed, and explicit states that make out-of-order or duplicate messages safe.
How did you handle duplicates from two route reflectors?	Both copies map to the same hash key. The second copy is recorded as an additional source with no new hardware work.
How does MAC mobility work?	The new location advertises with a higher sequence number. Everyone prefers the highest; the old owner withdraws. Rapid flapping is flagged as a duplicate.
How did you avoid hash-table slowdowns?	Pre-sized tables to avoid rehash during bursts, and a good custom hash that mixes VNI and MAC/IP fields to keep collisions low.
How was 40% measured?	Traffic-generator packet loss converted to outage time, across repeated failover tests on old and new builds, comparing medians and tail values.
What would you improve next?	Multi-threading per VNI shard, hardware-assisted prefix-independent convergence, and tighter BFD timers where the hardware supports it.